Autonomous Navigation: ROS2, SLAM, and TurtleBot3

1 Introduction to ROS (Robot Operating System)

Robot Operating System (ROS) is an open-source software framework designed to simplify the development of complex and robust robotic applications. Rather than being a traditional operating system, ROS provides a collection of tools, libraries, and conventions that enable modular and distributed robotics software development.

1.1 Core Concepts in ROS

1. **Node**: A fundamental process in ROS that performs a specific task. Each node is independent and communicates with other nodes to accomplish complex functions. Nodes can be written in various programming languages, such as Python or C++.
2. **Topic**: In ROS 2, a Topic is a unidirectional communication channel used by nodes to exchange data in a publish-subscribe pattern.
3. **Publisher**: A node that sends messages to a specific topic. Multiple nodes can publish to the same topic.
4. **Subscriber**: A node that receives messages from a topic. Subscribers can process or act upon the received data as needed.
5. **Services**: Topics are used to send continuous data, like sensor readings. Services, on the other hand, work more like asking a question and getting an answer. A node can send a request using a service and wait for a reply.

ROS uses a distributed graph architecture where nodes communicate via topics and services.

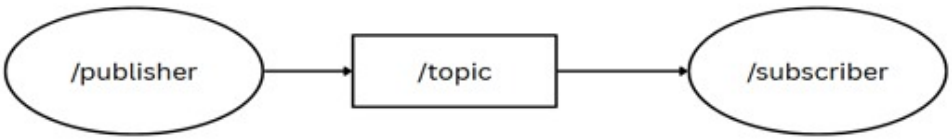

Figure 1: Communication between two nodes via topic.

2 Turtlesim

Turtlesim is a basic simulation tool in ROS 2 that visualizes a turtle moving in a window. It's mainly used to learn and practice ROS 2 concepts like nodes, topics, and services.

- **Launch Turtlesim:**

```
ros2 run turtlesim turtlesim_node
```

- **Control turtlesim by keyboard:**

```
ros2 run turtlesim turtle_teleop_key
```

2.1 rqt_graph

rqt_graph is a visualization tool in ROS that shows the communication between nodes using topics. It displays nodes as blocks and topics as arrows, helping developers understand data flow in a ROS system.

- **To open rqt_graph:**

```
rqt_graph
```

3 Creating a Workspace

A workspace is a directory where you develop, build, and install your ROS2 packages.

1. Create a workspace directory:

```
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws
```

2. Build the workspace:

```
colcon build
```

3. Source the environment:

```
source install/local_setup.bash
```

4 Creating a ROS 2 Package

A package is the basic unit of software in ROS 2. It contains code of nodes, launch files, dependencies, and parameters.

4.1 Creating a Package

```
cd ~/ros2_ws
ros2 pkg create --build-type ament_cmake my_package
```

For a Python package:

```
ros2 pkg create --build-type ament_python my_package
```

4.2 Package Structure

- `CMakeLists.txt`
- `package.xml`
- `include/`
- `src/` (e.g., `my_node.cpp`)
- `launch/` (e.g., `My_launch.py`)

4.3 Building and Using the Package

```
colcon build
source install/local_setup.bash
ros2 run my_package my_node
```

5 Creating Launch Files

Launch files let you start multiple nodes at once with custom configuration. They can be coded in XML, YAML, or Python.

5.1 Example XML Launch File

```
<?xml version="1.0" encoding="UTF-8"?>
<launch>
  <node pkg="turtlesim" exec="turtlesim_node" name="sim" namespace="turtlesim1" />
  <node pkg="turtlesim" exec="turtlesim_node" name="sim" namespace="turtlesim2" />
  <node pkg="turtlesim" exec="mimic" name="mimic">
    <remap from="/input/pose" to="/turtlesim1/turtle1/pose" />
    <remap from="/output/cmd_vel" to="/turtlesim2/turtle1/cmd_vel" />
  </node>
</launch>
```

Figure 2: This is my robot during navigation test

6 Creating a Node and a Subscriber

6.1 How to Create and Run a Node

1. Go into the respective directory (Package) and create the node file:

   ```
   touch <Node_Name.py>
   chmod +x <Node_Name.py>
   ```

2. Build the workspace:

   ```
   colcon build
   ```

3. Source the workspace:

   ```
   source install/setup.bash
   ```

4. Add the node file in `setup.py` to access it from any directory.

5. Run the node:

   ```
   ros2 run <package_name> <node_name>
   ```

```python
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from turtlesim.msg import Pose

class PoseSubscriberNode(Node):

    def __init__(self):
        super().__init__("pose_subscriber")
        self.pose_subscriber_ = self.create_subscription(
            Pose, "/turtle1/pose", self.pose_callback, 10)

    def pose_callback(self, msg: Pose):
        self.get_logger().info("(" + str(msg.x) + ", " + str(msg.y) + ")")

def main(args=None):
    rclpy.init(args=args)
    node = PoseSubscriberNode()
    rclpy.spin(node)
    rclpy.shutdown()

```

Figure 3: Subscriber Node

6.2 What is a Subscriber Node?

A subscriber node is a component in a publish-subscribe communication model, commonly used in systems like ROS. It waits for messages from a publisher node and communicates through topic, action, or service.

7 URDF in ROS 2: A Simple Introduction

URDF (Unified Robot Description Format) is an XML format used in ROS to describe a robot's physical configuration. It defines the robot's links, joints, visual elements, collision elements, and inertial properties. URDF is critical for simulation (Gazebo, Ignition), visualization (RViz), and robot control.

7.1 A Very Simple URDF Example

```
<?xml version="1.0"?>
<robot name="cylinder_bot">
  <link name="base_link">
    <visual>
      <geometry>
        <cylinder radius="0.1" length="0.3"/>
      </geometry>
      <origin xyz="0 0 0.15" rpy="0 0 0"/>
      <material name="gray">
        <color rgba="0.5 0.5 0.5 1"/>
      </material>
    </visual>
  </link>
</robot>
```

8 Gazebo

Gazebo is a robotics simulation software that allows you to simulate robots in complex environments with physics and sensors before testing in the real world.

- Simulate sensors
- Test robot control systems
- Simulate wheels, arms, and more parts
- Run tests without damaging hardware

8.1 Use of Gazebo in Autonomous Navigation

1. Creation of world: Create a 3D map with dynamic/static obstacles.
2. Visualization of bot: Design your bot in a URDF file and import it into Gazebo.
3. Simulation: Run the bot in the world with real-life physics simulation.
4. SLAM testing: Test mapping and localization in unknown environments.

9 Sensors Used in Autonomous Navigation

- **LiDAR**: Used for 3D mapping and obstacle detection.
- **Camera**: Visual SLAM, object classification, and mapping.
- **IMU**: Detects acceleration and rotation, useful for localization.
- **GPS**: Used for global positioning.
- **IR**: Used for close-range obstacle detection and avoidance.

10 Navigation: Cartographer

Cartographer is a real-time SLAM (Simultaneous Localization and Mapping) library that helps robots build a map of an unknown environment and track their own position.

10.1 How It Works

1. Takes input from sensors like LiDAR, IMU, etc.
2. Compares new sensor data with previous data.
3. Estimates movement, updates the robot's position, and builds a map.
4. Refines the map and corrects errors using loop closure.

10.2 Applications

- Drones and UAVs
- Indoor delivery robots
- Self-driving cars
- Search and rescue in unknown terrain

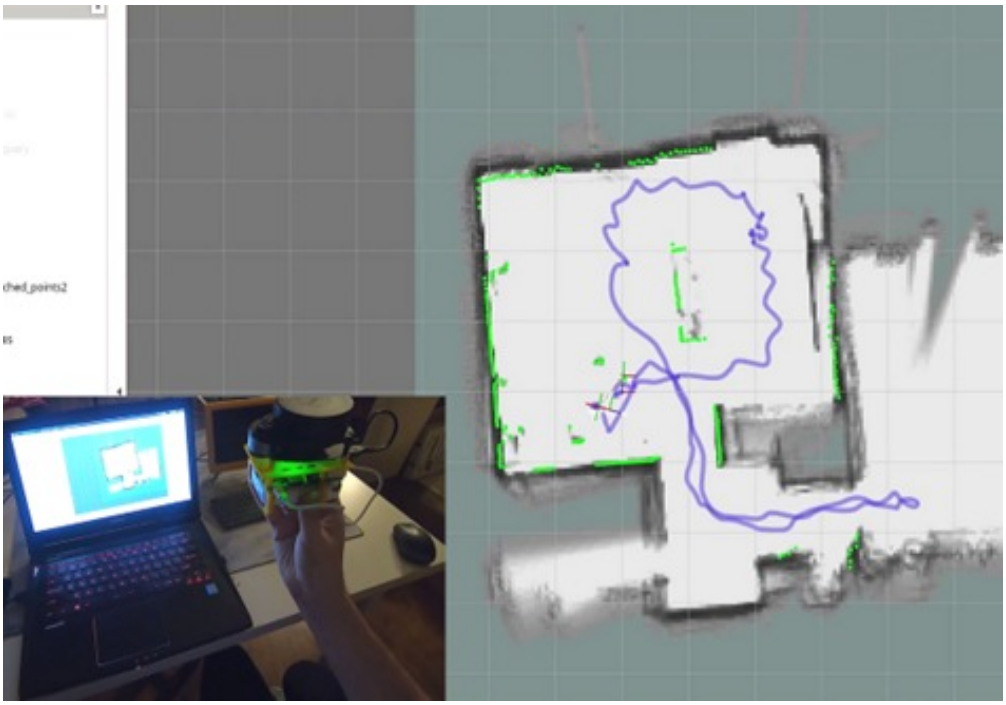

Figure 4: using cartographer for mapping

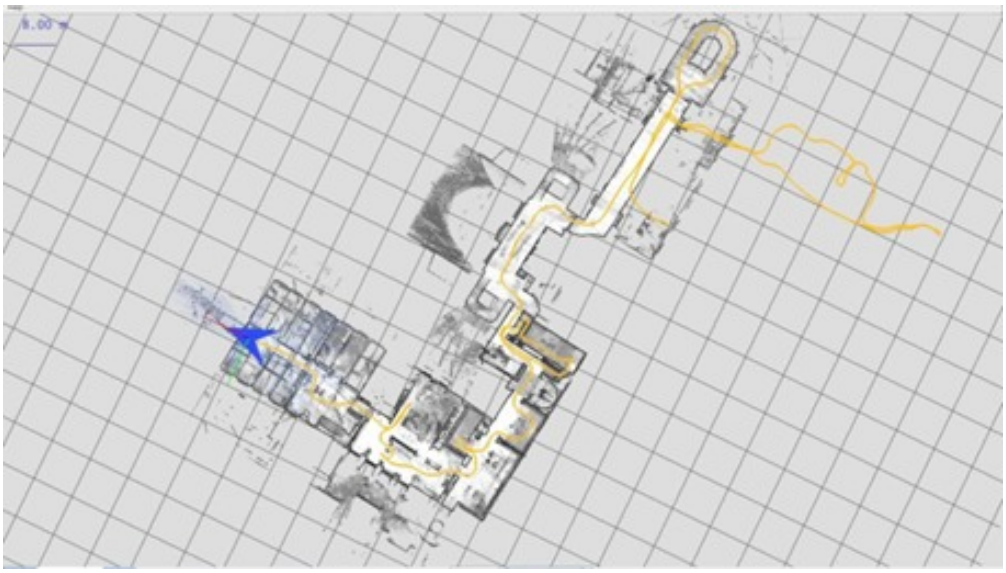

Figure 5: Output of cartographer

11 Sensor Fusion: LIDAR

- Combines LIDAR's 3D distance data with camera visuals for a richer, more accurate environmental view.
- Integrates the strengths of both sensors to overcome individual limitations.

11.1 Key Benefits

- Provides precise object detection and depth perception, even in low light or adverse weather.
- Enhances scene understanding by merging detailed shape (from LIDAR) and color/texture (from cameras).
- Increases safety and reliability by reducing ambiguity and sensor blind spots.
- Enables real-time processing for immediate decision-making.

11.2 Applications

- Widely used in autonomous vehicles for safe navigation, collision avoidance, and lane detection.
- Essential for robotics, allowing advanced obstacle avoidance and path planning.
- Supports smart mapping and infrastructure monitoring.

12 TurtleBot3

TurtleBot3 is a popular, low-cost, open-source robot developed by ROBOTIS and supported by the ROS community. It is widely used for learning, research, and prototyping in robotics.

- **Navigation**: Supports autonomous navigation using SLAM.
- **SLAM**: Can build a map using gmapping, Cartographer, or Hector SLAM.
- **Teleoperation**: Manual control via keyboard, joystick, or mobile app.
- **Simulation**: Fully supported in Gazebo.
- **Sensors and Hardware**: Includes Raspberry Pi or Intel SBC, LIDAR, IMU, and wheel encoders.
- **ROS Topics**: Common topics include `/cmd_vel`, `/scan`, `/odom`, and `/tf`.
- **Applications**: Robot programming, obstacle avoidance, mapping, multi-robot systems, and AI integration.

Figure 6: Robot model

13 SLAM

1. To solve the fundamental problem in robotics—navigating a dynamic environment with or without a provided map—we studied the concept of SLAM (Simultaneous Localization and Mapping).

2. The system must build a map of the unknown environment while simultaneously determining its own location within that map.

```
$ sudo apt install ros-humble-gazebo-*
$ sudo apt install ros-humble-cartographer
$ sudo apt install ros-humble-cartographer-ros

$ sudo apt install ros-humble-navigation2
$ sudo apt install ros-humble-nav2-bringup

$ source /opt/ros/humble/setup.bash
$ mkdir -p ~/turtlebot3_ws/src
$ cd ~/turtlebot3_ws/src/
$ git clone -b humble https://github.com/ROBOTIS-GIT/DynamixelSDK.git
$ git clone -b humble https://github.com/ROBOTIS-GIT/turtlebot3_msgs.git
$ git clone -b humble https://github.com/ROBOTIS-GIT/turtlebot3.git
$ sudo apt install python3-colcon-common-extensions
$ cd ~/turtlebot3_ws
$ colcon build --symlink-install
$ echo 'source ~/turtlebot3_ws/install/setup.bash' >> ~/.bashrc
$ source ~/.bashrc

$ echo 'export ROS_DOMAIN_ID=30 #TURTLEBOT3' >> ~/.bashrc
$ echo 'source /usr/share/gazebo/setup.sh' >> ~/.bashrc
$ echo 'source /opt/ros/humble/setup.bash' >> ~/.bashrc
$ source ~/.bashrc
```

Figure 7:

3. We implemented SLAM using Tortoise Bot and obtained the map.

```
$ export TURTLEBOT3_MODEL=burger
$ ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py
```

Figure 8:

```
$ export TURTLEBOT3_MODEL=burger
$ ros2 launch turtlebot3_cartographer cartographer.launch.py use_sim_time:=True
```

Figure 9:

```
$ export TURTLEBOT3_MODEL=burger
$ ros2 run turtlebot3_teleop teleop_keyboard

 Control Your TurtleBot3!
 ---------------------------
 Moving around:
        w
   a    s    d
        x

 w/x : increase/decrease linear velocity
 a/d : increase/decrease angular velocity
 space key, s : force stop

 CTRL-C to quit
```

Figure 10: Terminal to control bot

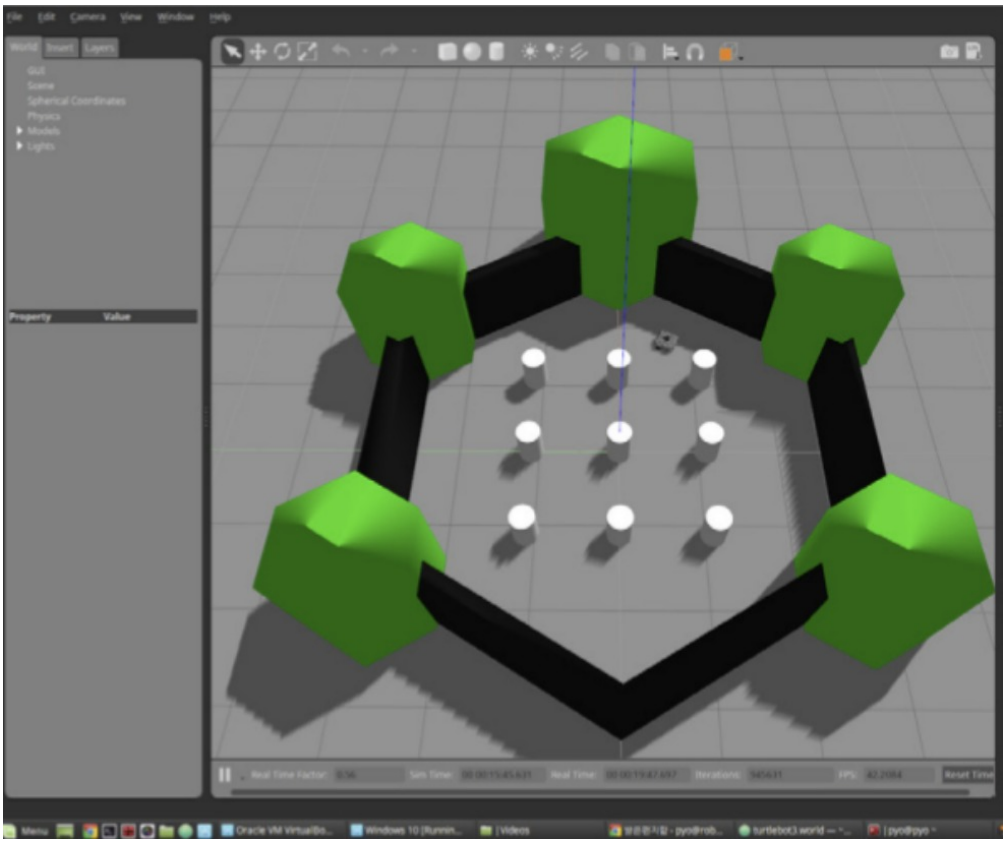

Figure 11: Demo input map from gazebo

The map obtained:

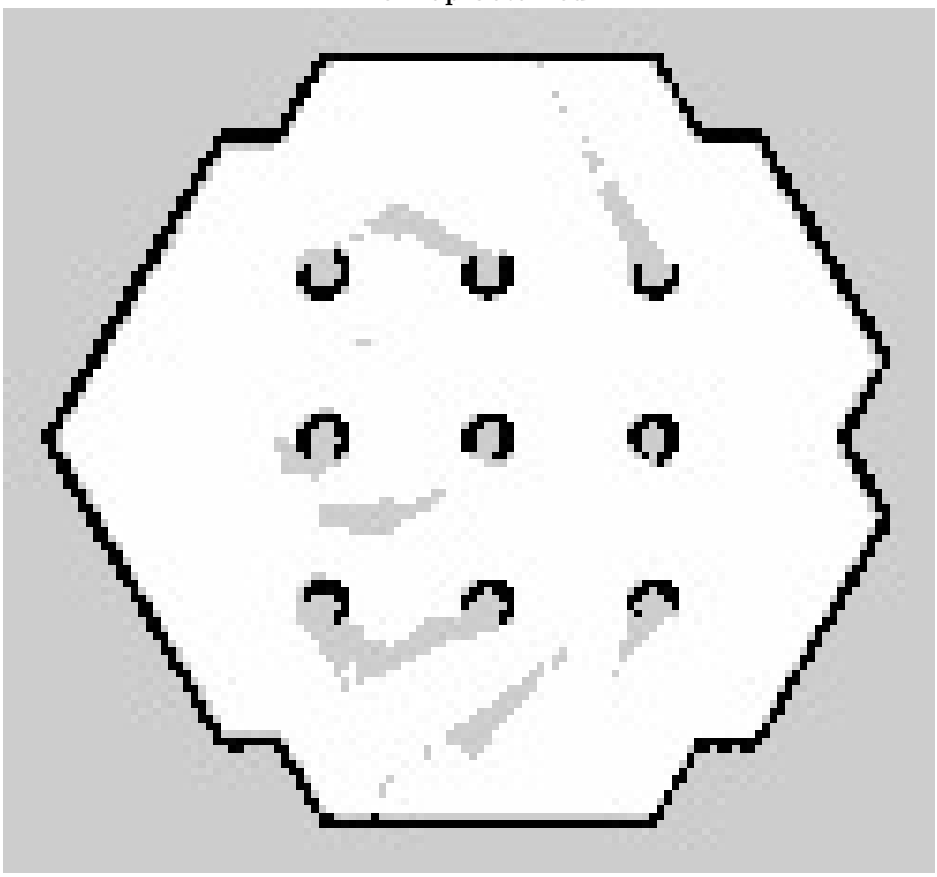

Figure 12: Final map